

Apply filters to SQL queries

Project description

By using SQL we can filter data from complex databases so that we can easily access the information that we need. This project will be used to demonstrate how to use SQL queries to filter out the data from the database.

The organization database contains the following two tables:

- log_in_attempts
- employees

log_in_attempts

The log_in_attempts table has the following columns:

- event_id: The identification number assigned to each login event
- username: The username of the employee
- login_date: The date the login attempt was recorded
- login_time: The time the login attempt was recorded
- country: The country where the login attempt occurred
- ip_address: The IP address of that employee's machine
- success: The success of the login attempt; FALSE indicates a failed attempt

In the MariaDB shell, these columns are returned as:

```
+-----+-----+-----+-----+-----+-----+-----+
| event_id | username | login_date | login_time | country | ip_address | success |
+-----+-----+-----+-----+-----+-----+-----+
```

employees

The employees table has the following columns:

- employee_id: The identification number assigned to each employee
- device_id: The identification number assigned to each device used by the employee
- username: The username of the employee
- department: The department the employee is in
- office: The office the employee is located in

In the MariaDB shell, these columns are returned as:

employee_id	device_id	username	department	office
-------------	-----------	----------	------------	--------

Retrieve after hours failed login attempts

Let's assume that we're investigating failed login attempts that were made after business hours, we'll identify all unsuccessful attempts after 18:00.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_time > '18:00' AND success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0
69	wjaffrey	2022-05-11	19:55:15	USA	192.168.100.17	0
82	abernard	2022-05-12	23:38:46	MEX	192.168.234.49	0
87	apatel	2022-05-08	22:38:31	CANADA	192.168.132.153	0
96	ivelasco	2022-05-09	22:36:36	CAN	192.168.84.194	0
104	asundara	2022-05-11	18:38:07	US	192.168.96.200	0
107	bisles	2022-05-12	20:25:57	USA	192.168.116.187	0
111	aestrada	2022-05-10	22:00:26	MEXICO	192.168.76.27	0
127	abellmas	2022-05-09	21:20:51	CANADA	192.168.70.122	0
131	bisles	2022-05-09	20:03:55	US	192.168.113.171	0
155	cgriffin	2022-05-12	22:18:42	USA	192.168.236.176	0
160	jclark	2022-05-10	20:49:00	CANADA	192.168.214.49	0
199	yappiah	2022-05-11	19:34:48	MEXICO	192.168.44.232	0

19 rows in set (0.149 sec)

As we can see there are 19 failed login attempts that occurred after 18:00.

Retrieve login attempts on specific dates

Let's assume a suspicious event occurred on 2022-05-09. To investigate this event, we want to review all login attempts which occurred on this day and the day before.

```

MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
+-----+-----+-----+-----+-----+-----+-----+
---+
| event_id | username | login_date | login_time | country | ip_address | success |
+-----+-----+-----+-----+-----+-----+-----+
---+
| 1 | jrafael | 2022-05-09 | 04:56:27 | CAN | 192.168.243.140 | 1 |
| 3 | dkot | 2022-05-09 | 06:47:41 | USA | 192.168.151.162 | 1 |
| 4 | dkot | 2022-05-08 | 02:00:39 | USA | 192.168.178.71 | 0 |
| 8 | bisles | 2022-05-08 | 01:30:17 | US | 192.168.119.173 | 0 |
| 12 | dkot | 2022-05-08 | 09:11:34 | USA | 192.168.100.158 | 1 |
| 15 | lyamamot | 2022-05-09 | 17:17:26 | USA | 192.168.183.51 | 0 |
| 24 | arusso | 2022-05-09 | 06:49:39 | MEXICO | 192.168.171.192 | 1 |
| 25 | sbaelish | 2022-05-09 | 07:04:02 | US | 192.168.33.137 | 1 |
| 172 | mabadi | 2022-05-08 | 08:06:50 | US | 192.168.180.41 | 1 |
| 178 | sgilmore | 2022-05-08 | 12:27:22 | CAN | 192.168.52.216 | 0 |
| 184 | alevitsk | 2022-05-08 | 03:09:48 | CAN | 192.168.33.70 | 0 |
| 186 | bisles | 2022-05-09 | 04:29:17 | USA | 192.168.40.72 | 0 |
| 187 | arusso | 2022-05-09 | 00:36:26 | MEX | 192.168.77.137 | 0 |
| 189 | nmason | 2022-05-08 | 05:37:24 | CANADA | 192.168.168.117 | 1 |
| 190 | jsoto | 2022-05-09 | 05:09:21 | USA | 192.168.25.60 | 0 |
| 191 | cjackson | 2022-05-08 | 06:46:07 | CANADA | 192.168.7.187 | 0 |
| 193 | lrodriqu | 2022-05-08 | 07:11:29 | US | 192.168.125.240 | 0 |
| 197 | jsoto | 2022-05-08 | 09:05:09 | US | 192.168.36.21 | 0 |
+-----+-----+-----+-----+-----+-----+-----+
---+
75 rows in set (0.001 sec)

```

As we can see from the terminal there were 75 log in attempts.

Retrieve login attempts outside of Mexico

There's been suspicious activity with login attempts, but the team has determined that this activity didn't originate in Mexico. To investigate this we need to use the query to filter only the countries outside of Mexico. The country column contains values of both MEX and MEXICO. In this case we'll need to use the LIKE keyword with % to make sure we get the correct information.

```
MariaDB [organization]> SELECT *
->
-> FROM log_in_attempts
->
-> WHERE NOT country LIKE 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
5	jrafael	2022-05-11	03:05:59	CANADA	192.168.86.232	0
7	eraab	2022-05-11	01:45:14	CAN	192.168.170.243	1
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
10	jrafael	2022-05-12	09:33:19	CANADA	192.168.228.221	0
11	sgilmore	2022-05-11	10:16:29	CANADA	192.168.140.81	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
13	mrhah	2022-05-11	09:29:34	USA	192.168.246.135	1
14	sbaelish	2022-05-10	10:20:18	US	192.168.16.99	1
184	alevitsk	2022-05-08	05:09:40	CAN	192.168.99.70	0
185	jsoto	2022-05-10	13:34:58	USA	192.168.151.91	0
186	bisles	2022-05-09	04:29:17	USA	192.168.40.72	0
188	jsoto	2022-05-11	00:39:09	USA	192.168.21.88	0
189	nmason	2022-05-08	05:37:24	CANADA	192.168.168.117	1
190	jsoto	2022-05-09	05:09:21	USA	192.168.25.60	0
191	cjackson	2022-05-08	06:46:07	CANADA	192.168.7.187	0
192	bisles	2022-05-10	08:32:03	USA	192.168.201.40	1
193	lrodriqu	2022-05-08	07:11:29	US	192.168.125.240	0
194	jclark	2022-05-12	14:11:04	CAN	192.168.197.247	0
195	alevitsk	2022-05-11	06:59:13	CANADA	192.168.236.78	1
196	acook	2022-05-10	09:56:48	CAN	192.168.52.90	0
197	jsoto	2022-05-08	09:05:09	US	192.168.36.21	0
200	jclark	2022-05-12	01:11:45	CANADA	192.168.91.103	1

```
144 rows in set (0.001 sec)
```

As we can see there were total of 144 login attempts made outside of Mexico.

Retrieve employees in Marketing

Let's say that we want to perform security updates on specific employee machines in the Marketing department who are located in all offices in the East building (such as 'East-170' or 'East-320'). The department of the employee is found in the **department** column, which contains values that include **Marketing**. The office is found

in the office column. Some examples of values in this column are **East-170**, **East-320**, and **North-434**. We'll need to use the **LIKE** keyword with % to filter for the East building.)

```
MariaDB [organization]> SELECT *
->
-> FROM employees
->
-> WHERE department = 'Marketing' AND office LIKE 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267
1088	k865l965m233	rgosh	Marketing	East-157
1103	NULL	randeross	Marketing	East-460
1156	a184b775c707	dellery	Marketing	East-417
1163	h679i515j339	cwilliam	Marketing	East-216

```
7 rows in set (0.034 sec)
```

Retrieve employees in Finance or Sales

Now, let's say that the team needs to perform a different update to the computers of all employees in the Finance or the Sales department, and we need to locate information on these employees. We should use the **OR** operator to accomplish this.

```
MariaDB [organization]> SELECT *
->
-> FROM employees
->
-> WHERE department = 'Finance' OR department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134

Retrieve all employees not in IT

The team needs to make one more update. This update was already made to employee computers in the Information Technology department. The team needs information about employees who are not in that department. We should use the **NOT** operator to identify these employees.

```
MariaDB [organization]> SELECT *  
->  
-> FROM employees  
->  
-> WHERE NOT department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1016	q793r736s288	sbaelish	Human Resources	North-229
1017	r550s824t230	jclark	Finance	North-188

Summary

This project demonstrated how operators like **AND**, **OR**, **NOT**, and **LIKE** are essential for investigating security incidents. By using them, I was able to isolate failed after-hours logins, review activity from specific dates and locations, and gather information for targeted system updates, showcasing how SQL is a critical tool for any security professional.